Course: Cloud Computing Lab **Course Code**: CS402

Sem: VII Sem CSE Date of submission:

Introduction to Docker: Creating and Running a Containerized Web Application on Windows

Aim
To install and configure Docker Desktop on Windows and create, run, inspect, stop, and remove a Docker container using the Nginx web server.

Tasks:
Install Docker Desktop → verify Docker → pull an image → create/run a container → access a web application → stop/remove the container.

Docker's current Windows documentation recommends Docker Desktop with the **WSL 2 backend** for most Windows users. Current requirements include supported Windows editions/builds, WSL 2, a 64-bit processor with SLAT, 8 GB RAM, and hardware virtualization enabled.

Docker Desktop Installation on Windows

Step 1: Download Docker Desktop

Download Docker Desktop from the official Docker website:

https://docs.docker.com/desktop/setup/install/windows-install/

Step 2: Install Docker Desktop

Run:

```
Docker Desktop Installer.exe
```

During installation, select the **WSL 2** option when prompted.

Docker currently recommends the per-user installation mode for most users, and WSL 2 is the default backend for typical Windows installations.

Step 3: Start Docker Desktop

After installation:

```
Start Menu
     ↓
Docker Desktop
```

Wait until Docker Desktop indicates that Docker is running.

Verify Docker Installation

Open **Command Prompt** or **PowerShell**.

Execute:

```
docker --version
```

Example output:

```
Docker version 28.x.x
```

The exact version will depend on the version installed on the laboratory computer.

Now execute:

```
docker info
```

If Docker is running correctly, information about the Docker Engine will be displayed.

Experiment 1 – Run Hello World Container

Before running the web server, students should verify that containers can be executed.

Execute:

```
docker run hello-world
```

Docker downloads the `hello-world` image if it is not already available locally, creates a container, runs it, displays a confirmation message, and exits.

Expected Result

The terminal should display a message similar to:

```
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

Observation

```
Docker Client
     ↓
Docker Engine
     ↓
Docker Hub
     ↓
hello-world Image
     ↓
Container
     ↓
Output
```

Experiment 2:

- **docker --version**
- **docker create nginx (or you can use docker pull nginx)**
- **docker run nginx:latest**
- **Using notepad create a file called dockerfile** with the following commands:

FROM python:3.12
WORKDIR /app
COPY app.py .
CMD[“python”, “app.py”]

- **docker build -t my-python-app .**
- **docker images**
- **docker run my-python-app**

You should see the message “Hello from my Python application!”
Take 1 or more screenshots of all the above steps.

Experiment 3 – Run an Nginx Web Server

This is the main experiment.

Step 1: Download Nginx Image

Execute:

```
docker pull nginx
```

This downloads the Nginx image from the container registry.

Step 2: Verify the Image

Execute:

```
docker images
```

You should see an entry similar to:

```
REPOSITORY    TAG       IMAGE ID       CREATED       SIZE
nginx         latest    xxxxxxxxxxxx   ...           ...
```

Step 3: Create and Run the Container

Execute:

```
docker run -d --name mynginx -p 8080:80 nginx
```

Explanation

Command/Option	Meaning
`docker run`	Create and run a container
`-d`	Run container in detached/background mode
`--name mynginx`	Assign the name `mynginx`
`-p 8080:80`	Map host port 8080 to container port 80
`nginx`	Docker image to use

The important concept is:

```
Windows Host                    Container
Port 8080       ──────────────→ Port 80
                                  |
                                  ↓
                               Nginx Server
```

Step 4: Check the Running Container

Execute:

```
docker ps
```

Expected output will contain something similar to:

```
CONTAINER ID   IMAGE   COMMAND                  STATUS
xxxxxxxxxxxx   nginx   "/docker-entrypoint..."  Up ...
```

The container should have a status similar to:

```
Up ...
```

Step 5: Access the Web Application

Open a web browser and enter:

```
http://localhost:8080
```

The Nginx welcome page should appear.

Expected Result

```
Welcome to nginx!
```

This demonstrates that a web server running **inside a Docker container** can be accessed from the Windows host through port mapping.

Step 6: View Container Logs

Execute:

```
docker logs mynginx
```

Docker will display the logs generated by the Nginx container.

Students can refresh:

```
http://localhost:8080
```

and then execute:

```
docker logs mynginx
```

again to observe HTTP request entries.

Step 7: Stop the Container

Execute:

```
docker stop mynginx
```

Check:

```
docker ps
```

The container will no longer appear among running containers.

To see stopped containers:

```
docker ps -a
```

Step 8: Start the Container Again

Instead of creating a new container, start the existing one:

```
docker start mynginx
```

Verify:

```
docker ps
```

Then open:

```
http://localhost:8080
```

The Nginx page should appear again.

Step 9: Remove the Container

First stop it:

```
docker stop mynginx
```

Then remove it:

```
docker rm mynginx
```

Verify:

```
docker ps -a
```

The `mynginx` container should no longer be listed.

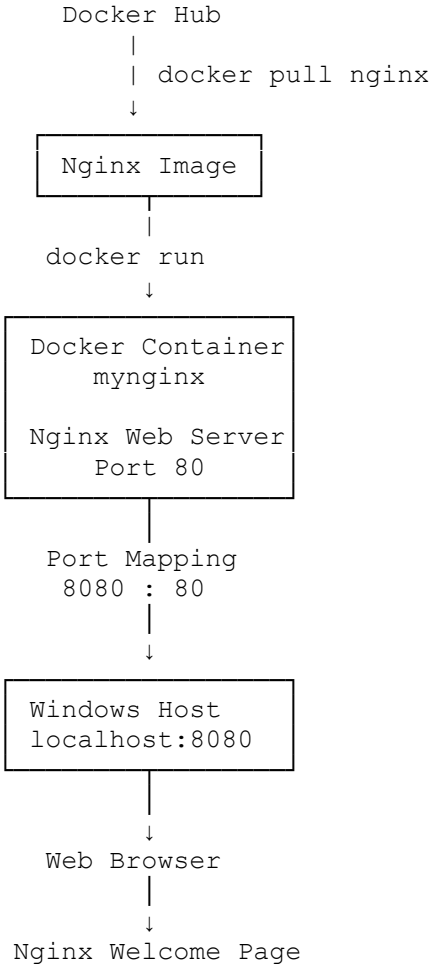
Important Docker Commands

Students should get familiar with the following commands.

Command	Purpose
`docker --version`	Check Docker version
`docker info`	Display Docker Engine information
`docker pull nginx`	Download Nginx image
`docker images`	List Docker images

Command	Purpose
`docker run nginx`	Create and run Nginx container
`docker ps`	List running containers
`docker ps -a`	List all containers
`docker logs mynginx`	Display container logs
`docker stop mynginx`	Stop container
`docker start mynginx`	Start stopped container
`docker rm mynginx`	Remove container
`docker rmi nginx`	Remove Docker image

Docker Architecture for This Experiment



Screenshots to Include for Experiment 3

Students should capture the following screenshots:

Screenshot 1: Docker Desktop running

Screenshot 2:

```
docker --version
```

Screenshot 3:

```
docker run hello-world
```

showing successful execution.

Screenshot 4:

```
docker images
```

showing the Nginx image.

Screenshot 5:

```
docker ps
```

showing `mynginx` running.

Screenshot 6: Browser showing:

```
http://localhost:8080
```

and the Nginx welcome page.

Screenshot 7:

```
docker logs mynginx
```

Screenshot 8:

```
docker ps -a
```

after stopping/removing the container.
